University of Information Technology and Sciences (UITS)
Department of CSE

Simulation and Modeling Lab
Course Code: CSE-413

Weather Simulation & Statistical Analysis

By

Md. Mefazur Rahman	ID: 0432310005101035
Nazmul Chowdury	ID: 0432310005101044
Md. Alif Reza Shimanto	ID: 0432310005101046
Md. Shoyaif Rahman	ID: 0432310005101050
Batch :53	Section:7A

Department of CSE

Submitted to

Afroja Ahmed Smrity

Lecturer

Department of CSE, UITS

Baridhara J Block, Dhaka 1212

28 June, 2026

Table of Contents

1. Introduction

1.1 Background

The weather is something that's very hard to predict. This is because there are many things that can affect it. Simulation is a way to study the weather. It does this by creating data that is similar to real data. This lets us look at patterns and test ideas without having to wait for a time to get real data.

This project is about temperature. It uses something called the distribution to create fake temperature readings for thirty days in two cities in Bangladesh. Dhaka and Cox's Bazar. Then it uses tests to see if the differences in the fake data are real or just happened by chance. These tests are called a one-sample t-test, a two-sample t-test and a square goodness-of-fit test. The project uses Python and some special tools like NumPy, SciPy and Matplotlib to make the data test the ideas and make pictures of the data. It even makes an animated picture that shows the temperature, for each day.

1.2 Problem Statement

Temperature conditions vary noticeably between cities located in different geographic settings. Dhaka, a dense inland metropolitan city, and Cox's Bazar, a coastal town, are commonly assumed to experience different thermal conditions, but confirming such a difference informally is not a sound basis for a quantitative claim. The problem this project addresses is twofold: (1) how to generate a realistic, reproducible synthetic temperature dataset for two cities, and (2) how to rigorously determine — through hypothesis testing rather than visual impression — whether the temperature of one city differs significantly from a reference value and from the temperature of the other city.

1.3 Objectives

- To simulate 30 days of daily temperature data for Dhaka and Cox's Bazar using the Normal probability distribution.
- To compute descriptive statistics (mean, standard deviation, minimum, maximum, median) for each simulated series.
- To test, using a one-sample t-test, whether Dhaka's simulated mean temperature differs significantly from a national reference average of 30°C.
- To test, using a two-sample t-test, whether the mean temperatures of Dhaka and Cox's Bazar differ significantly from each other.

- To test, using a chi-square goodness-of-fit test, whether Dhaka's daily temperatures are evenly distributed across Cool, Warm, and Hot categories.
- To visualize the simulated data and the test results through static charts and a day-by-day animation.

1.4 Scope of the Project

This project is about the temperature in two cities over thirty days. It uses the distribution and three hypothesis tests to do this. The project does not use information and it does not look at other weather things like rain or humidity. It also does not try to guess what the weather will be like in the future. The project is meant to show students how to use simulation and statistics, in Python. The project talks about how it could be made better in the future like using information to make it more realistic.

2. Literature Review

Stochastic generation of synthetic weather data has a long history in hydrology, agriculture, and climate-impact studies. Richardson [1] proposed one of the earliest and most influential stochastic weather generators, modelling daily precipitation occurrence with a Markov chain and conditioning temperature and solar radiation on the wet or dry state of each day through a multivariate model fitted from historical station records. Building on this and related work, Wilks and Wilby [2] provide a comprehensive review of stochastic weather models, tracing their development from single-site rainfall-occurrence models to multisite generators capable of reproducing the statistical structure of historical climate records.

These established weather generators are considerably more elaborate than the model used in this project: they explicitly condition temperature on precipitation state and calibrate their distribution parameters from years of observed station data, whereas the present work draws temperature directly from two independent Normal distributions with fixed, illustrative parameters. The simplification is deliberate — the purpose here is pedagogical, to demonstrate the mechanics of simulation and hypothesis testing, rather than to produce a climatologically calibrated generator.

On the inferential side, the one-sample t-test, the two-sample t-test, and the chi-square goodness-of-fit test used in this project are classical procedures described in standard engineering-statistics texts such as Montgomery and Runger [6]. They remain the standard first tools for comparing a sample mean against a reference value, comparing two independent sample means, and testing whether observed categorical counts match an expected distribution.

The simulation and analysis were implemented entirely in Python, using NumPy [3] for random-number generation and array operations, SciPy [4] for the t-test and chi-square procedures, and Matplotlib [5] for static plotting and the FuncAnimation-based animation. Together, these libraries form the de-facto standard scientific-computing stack for exactly this kind of small-scale simulation and statistical-analysis task in an academic setting, and their use here follows the conventions established in the wider stochastic-weather-generation literature reviewed above.

3. Dataset and Feature Details

Unlike most data-analysis projects, this project does not consume an externally collected dataset. Instead, the dataset is generated programmatically inside the simulation itself, which has the advantage of being fully reproducible (through a fixed random seed) and free of missing values, sensor noise, or recording errors.

3.1 Generation Parameters

Two independent 30-day temperature series are drawn from Normal distributions with the following parameters:

- Dhaka: mean $\mu = 32°C$, standard deviation $\sigma = 3°C$
- Cox's Bazar: mean $\mu = 27°C$, standard deviation $\sigma = 4°C$

NumPy's pseudo-random generator is seeded with random_seed = 42 before either series is generated, so that the same dataset — and consequently the same test statistics reported in this document — can be reproduced exactly on any machine running the same code.

3.2 Feature Description

Table 1: *Description of the features present in the simulated dataset.*

Feature	Type	Description
Day	Integer (1–30)	Sequential day index of the 30-day simulation window.
Dhaka_Temperature	Continuous (°C)	Simulated daily temperature for Dhaka, drawn from $N(\mu=32, \sigma=3)$.
CoxsBazar_Temperature	Continuous (°C)	Simulated daily temperature for Cox's Bazar, drawn from $N(\mu=27, \sigma=4)$.
Temperature_Category (derived)	Categorical: {Cool, Warm, Hot}	Cool < 28°C, Warm 28–32°C, Hot > 32°C — derived from Dhaka_Temperature for the chi-square test.

3.3 Descriptive Statistics of the Generated Sample

Table 2: *Sample descriptive statistics computed from the simulated 30-day series*

City	N	Mean (°C)	Std. Dev (°C)	Min (°C)	Max (°C)	Median (°C)
Dhaka	30	31.44	2.70	26.26	36.74	31.30
Cox’s Bazar	30	26.52	3.72	19.16	34.41	26.74

Although the underlying generating parameters are $\mu = 32$°C, $\sigma = 3$°C for Dhaka and $\mu = 27$°C, $\sigma = 4$°C for Cox’s Bazar, the realized 30-day sample statistics differ slightly from these population values (e.g., 31.44°C rather than exactly 32°C for Dhaka) because any finite random sample only approximates its parent distribution. This sampling variability is precisely why hypothesis testing — rather than a simple comparison of the generating parameters — is required to draw a statistically defensible conclusion, and it motivates the methodology described in the next section.

4. Methodology with Equations and Block Diagram

This section describes the end-to-end methodology used to generate the simulated dataset and to statistically analyse it, together with the governing equations for each step. Table 3 lists the tools and technologies used throughout the pipeline.

Table 3: *Tools and technologies used in the simulation pipeline.*

Tool / Technology	Purpose
Python 3	Core programming language for the entire pipeline.
NumPy	Random-number generation (Normal distribution) and array operations.
SciPy (scipy.stats)	One-sample t-test, two-sample t-test, chi-square critical values, confidence intervals.
Matplotlib	Static visualization and FuncAnimation-based day-by-day animation.
Google Colab	Development and execution environment.

4.1 System Overview and Block Diagram

Figure 1 shows the overall processing pipeline used in this project. The input parameters — the two Normal-distribution specifications and the fixed random seed — are passed into a simulation engine built on NumPy's random-number generator, which produces the raw 30-day temperature series for each city. These series are then passed to a statistical-analysis stage that performs the three hypothesis tests described below; the raw data and the test outputs are subsequently passed to a visualization stage that produces the static charts and the day-by-day animation, after which the numeric results and plots are assembled into the output report.

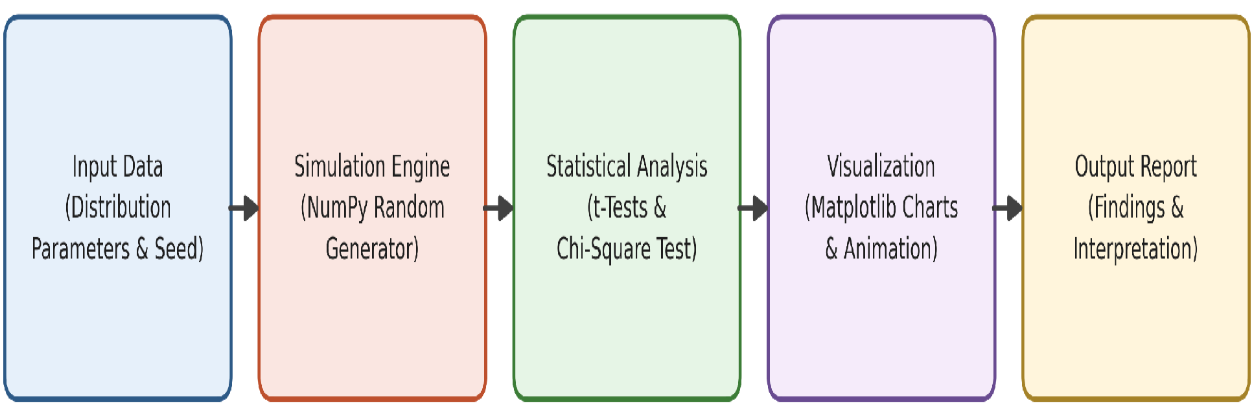

Figure 1: *System architecture (processing pipeline) of the weather simulation system.*

4.2 Data Generation Model

Each daily temperature observation is treated as an independent draw from a Normal (Gaussian) distribution. Its probability density function is given in Equation 1, where μ is the population mean and σ is the population standard deviation of the city's temperature.

$$f(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Equation 1: *Probability density function of the Normal distribution used to generate each city's daily temperatures.*

4.3 Descriptive Statistics

For a sample of n daily readings $x_1, x_2, \ldots, x_n$, the sample mean and sample standard deviation are computed as shown in Equation 2. These two quantities summarize the central tendency and spread of each city's simulated series and feed directly into the hypothesis tests that follow.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad\qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Equation 2: *Sample mean and sample standard deviation.*

4.4 One-Sample t-Test

The one-sample t-test checks whether Dhaka's mean temperature differs significantly from a hypothesized national reference average of 30°C.

H_0 (null hypothesis): the mean temperature of Dhaka equals 30°C.

H_1 (alternative hypothesis): the mean temperature of Dhaka is not equal to 30°C.

The test statistic and its degrees of freedom are computed using Equation 3, and the 95% confidence interval for the true mean is computed using Equation 4. If the resulting p-value is less than the significance level $\alpha = 0.05$, the null hypothesis is rejected.

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}\ , \qquad df = n - 1$$

Equation 3: *One-sample t-statistic and degrees of freedom.*

$$CI_{95\%} = \bar{x} \pm t_{\alpha/2,\, df}\left(\frac{s}{\sqrt{n}}\right)$$

Equation 4: *95% confidence interval for the population mean.*

4.5 Two-Sample t-Test

The two-sample (independent, equal-variance) t-test compares the mean temperatures of Dhaka and Cox's Bazar.

H_0: the mean temperatures of Dhaka and Cox's Bazar are equal.

H_1: the mean temperatures of Dhaka and Cox's Bazar are significantly different.

Because both samples are assumed to have equal population variance, a pooled variance estimate is first computed using Equation 5, and the t-statistic is then computed using Equation 6 with $n_1 + n_2 - 2$ degrees of freedom. As with the one-sample test, a p-value below 0.05 leads to rejection of H_0.

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

Equation 5: *Pooled sample variance for two independent groups.*

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_p^2\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}, \qquad df = n_1 + n_2 - 2$$

Equation 6: *Two-sample (pooled-variance) t-statistic and degrees of freedom.*

4.6 Chi-Square Goodness-of-Fit Test

The chi-square goodness-of-fit test checks whether Dhaka's 30 simulated days are evenly distributed across three temperature categories: Cool (< 28°C), Warm (28–32°C), and Hot (> 32°C).

H_0: all three categories occur with equal frequency (a uniform distribution).

H_1: at least one category occurs significantly more or less often than expected.

Under H_0, each of the three categories is expected to contain $n/3 = 10$ of the 30 days. The chi-square statistic is computed using Equation 7 with $k - 1 = 2$ degrees of freedom, where Oi and Ei are the observed and expected counts for category i. H_0 is rejected if the computed statistic exceeds the critical value at $\alpha = 0.05$ (equivalently, if the p-value is below 0.05).

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i} \,, \qquad df = k - 1$$

Equation 7: *Chi-square goodness-of-fit statistic.*

4.7 Visualization and Animation

In addition to the numerical hypothesis tests, Matplotlib is used to render a static line chart of both 30-day series, paired bar/error-bar charts for the one-sample and two-sample comparisons, a grouped bar chart for the chi-square observed-versus-expected counts, and a FuncAnimation-based animation that reveals the two temperature curves one day at a time. The animation does not alter the statistical conclusions; it is included to make the day-to-day fluctuation in temperature more intuitive to a viewer than a static chart alone.

5. Pseudocode

```
ALGORITHM Weather Simulation And Analysis
INPUT : mu_D = 32, sigma_D = 3, mu_C = 27, sigma_C = 4,
        n = 30, seed = 42, alpha = 0.05
OUTPUT: hypothesis-test results, charts, animation

1.  SET random_seed ← 42

2.  // Step 1: Data Simulation
    Dhaka[1..n]     ← NORMAL(mean = mu_D, std = sigma_D, size = n)
    CoxsBazar[1..n] ← NORMAL(mean = mu_C, std = sigma_C, size = n)
    Days[1..n]      ← 1, 2, ..., n

3.  // Step 2: Descriptive Statistics
    mean_D, std_D ← MEAN(Dhaka), STD(Dhaka)
    mean_C, std_C ← MEAN(CoxsBazar), STD(CoxsBazar)

4.  // Step 3: One-Sample t-Test   (H0: mean_D = 30)
    t1, p1 ← ONE_SAMPLE_T_TEST(Dhaka, mu0 = 30)
    CI1    ← T_CONFIDENCE_INTERVAL(Dhaka, confidence = 0.95)
    IF p1 < alpha THEN REJECT H0_1 ELSE FAIL_TO_REJECT H0_1

5.  // Step 4: Two-Sample t-Test   (H0: mean_D = mean_C)
    t2, p2 ← TWO_SAMPLE_T_TEST(Dhaka, CoxsBazar, equal_var = TRUE)
    CI2    ← DIFFERENCE_CONFIDENCE_INTERVAL(Dhaka, CoxsBazar)
    IF p2 < alpha THEN REJECT H0_2 ELSE FAIL_TO_REJECT H0_2

6.  // Step 5: Chi-Square Goodness-of-Fit Test
    FOR each temp IN Dhaka:
        CATEGORIZE temp AS Cool (<28), Warm (28-32), or Hot (>32)
    Observed[1..3] ← COUNT(Cool), COUNT(Warm), COUNT(Hot)
    Expected[1..3] ← n/3, n/3, n/3
    chi_stat       ← SUM( (Observed[i] - Expected[i])^2 / Expected[i] )
    critical_value ← CHI_SQUARE_INVERSE(0.95, df = 2)
    IF chi_stat > critical_value THEN REJECT H0_3 ELSE FAIL_TO_REJECT H0_3

7.  // Step 6: Visualization
    PLOT line chart of Dhaka & CoxsBazar vs Days
    PLOT bar / box charts for each test
    ANIMATE day-by-day temperature progression (31 frames)

8.  // Step 7: Reporting
    PRINT all statistics, p-values, confidence intervals, decisions
    RETURN results
END ALGORITHM
```

6. Result Analysis & Discussion

6.1 Simulated Temperature Data

Figure 2 shows the simulated 30-day temperature series for both cities. Dhaka's curve sits above Cox's Bazar's curve on most days and stays close to or above the 30°C reference line, while Cox's Bazar shows a visibly wider and cooler spread, including several days that drop well below 25°C. The two curves do cross on a handful of days, which is expected given the overlap between N(32, 3) and N(27, 4), but the overall separation between the two series is visually apparent even before any formal test is applied.

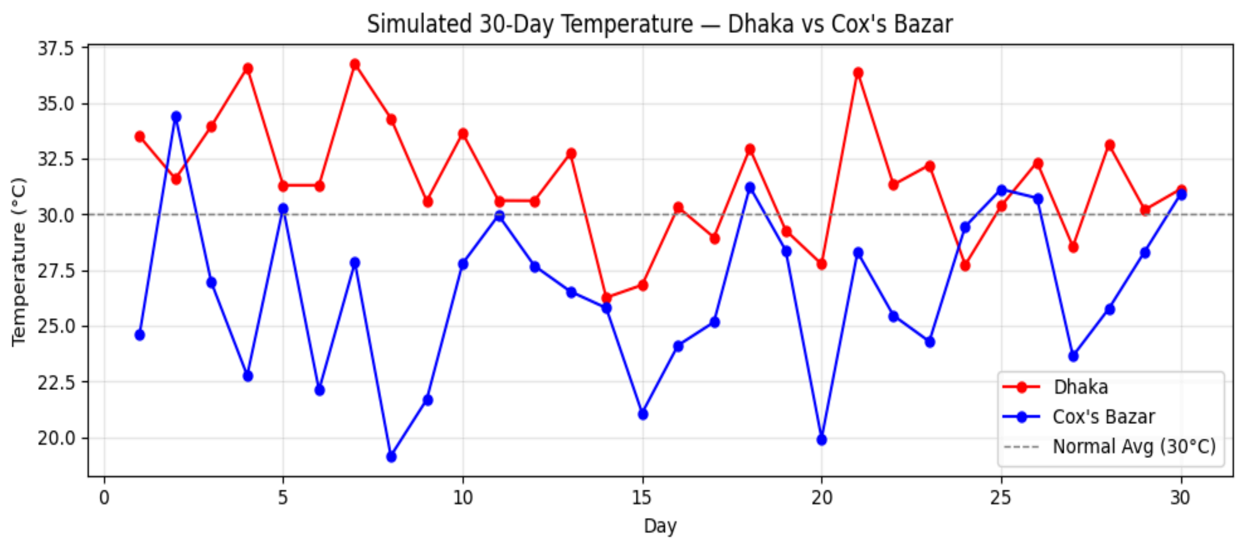

Figure 2: *Simulated 30-day temperature series — Dhaka vs. Cox's Bazar.*

6.2 One-Sample t-Test Results

Table 4 summarizes the one-sample t-test comparing Dhaka's simulated mean temperature against the 30°C national reference average; Figure 3 shows the same comparison graphically, with the error bar representing one standard deviation.

Table 4: One-sample t-test summary (Dhaka vs. 30°C reference).

Statistic	Value
Sample Mean (Dhaka)	31.44 °C
Hypothesized Mean (H_0)	30.00 °C
t-statistic	2.9122
Degrees of Freedom	29
p-value	0.0068
95% CI for the mean	[30.43, 32.44] °C
Decision ($\alpha = 0.05$)	Reject H_0

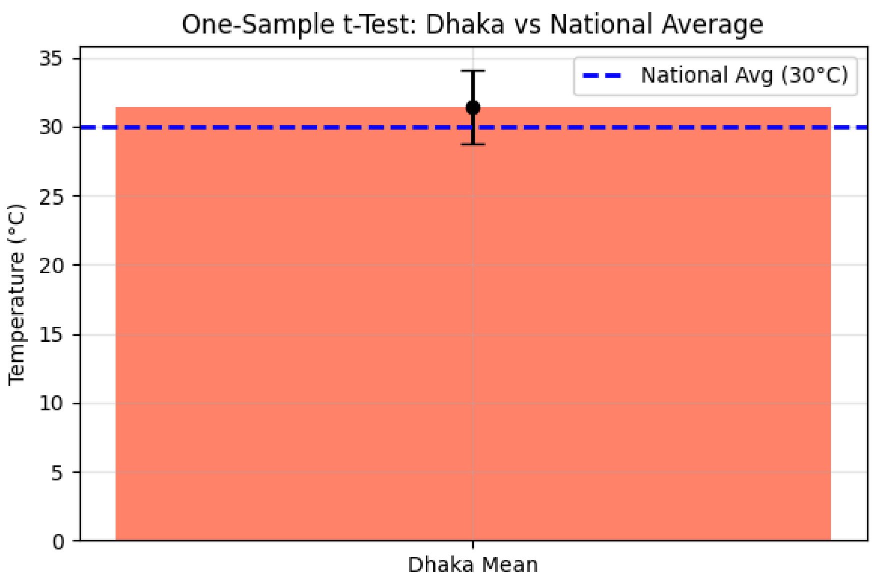

***Figure 3:** One-sample t-test — Dhaka's mean vs. the national average (30°C).*

With $p = 0.0068$, which is well below $\alpha = 0.05$, the null hypothesis is rejected: the simulated data provide statistically significant evidence that Dhaka's mean temperature is higher than 30°C. The 95% confidence interval, [30.43°C, 32.44°C], does not contain 30°C, which is consistent with this conclusion.

6.3 Two-Sample t-Test Results

Table 5 summarizes the two-sample t-test comparing the mean temperatures of Dhaka and Cox's Bazar; Figure 4 shows both a mean-comparison bar chart and a box plot of the two distributions.

__Table 5:__ Two-sample t-test summary (Dhaka vs. Cox's Bazar).

Statistic	Value
Dhaka Mean	31.44 °C
Cox's Bazar Mean	26.52 °C
Mean Difference	4.92 °C
t-statistic	5.8583
Degrees of Freedom	58
p-value	< 0.0001
95% CI for the difference	[3.24, 6.60] °C
Decision ($\alpha = 0.05$)	Reject H_0

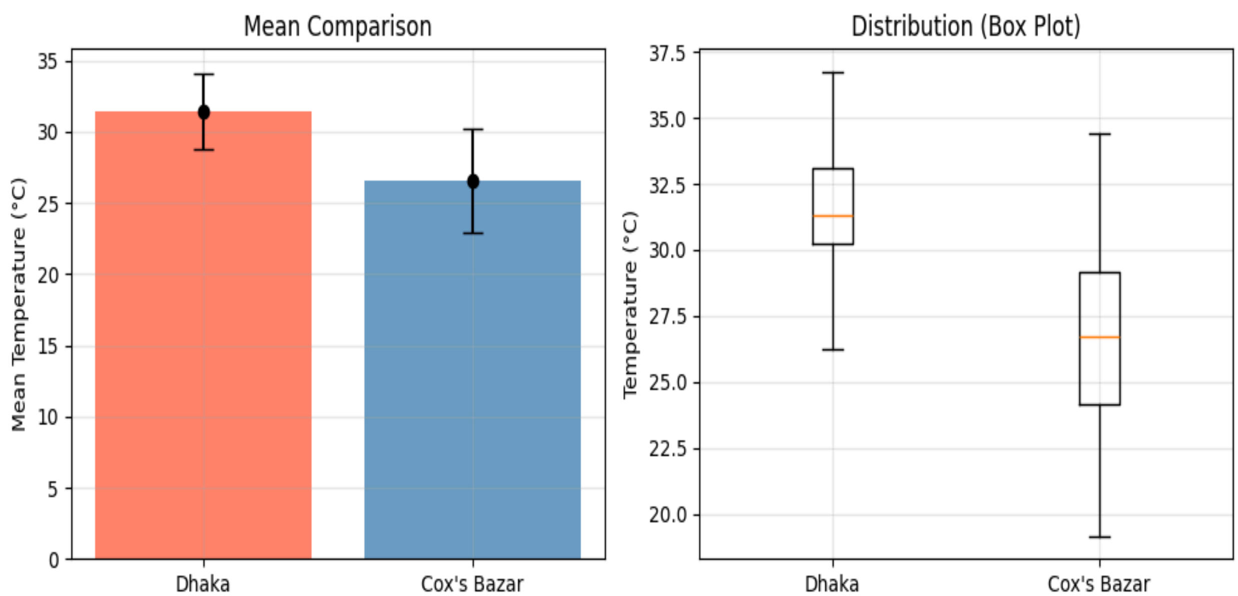

Figure 4: *Two-sample comparison of Dhaka and Cox's Bazar — mean ± standard deviation (left) and box plot (right).*

The p-value is well below 0.0001, so the null hypothesis of equal means is rejected with a very high degree of confidence. The 95% confidence interval for the mean difference, [3.24°C, 6.60°C], lies entirely above zero, confirming that Dhaka is, on average, meaningfully and significantly hotter than Cox's Bazar in the simulated data — consistent with the box plot, in which Dhaka's entire interquartile range sits above Cox's Bazar's median.

6.4 Chi-Square Test Results

Table 6 and Figure 5 summarize the chi-square goodness-of-fit test on Dhaka's temperature categories.

Table 6: *Chi-square goodness-of-fit summary for Dhaka's temperature categories.*

Category	Observed (O)	Expected (E)	$(O - E)^2 / E$
Cool (< 28°C)	4	10.0	3.60
Warm (28–32°C)	14	10.0	1.60
Hot (> 32°C)	12	10.0	0.40
Total	30	30.0	$\chi^2 = 5.60$

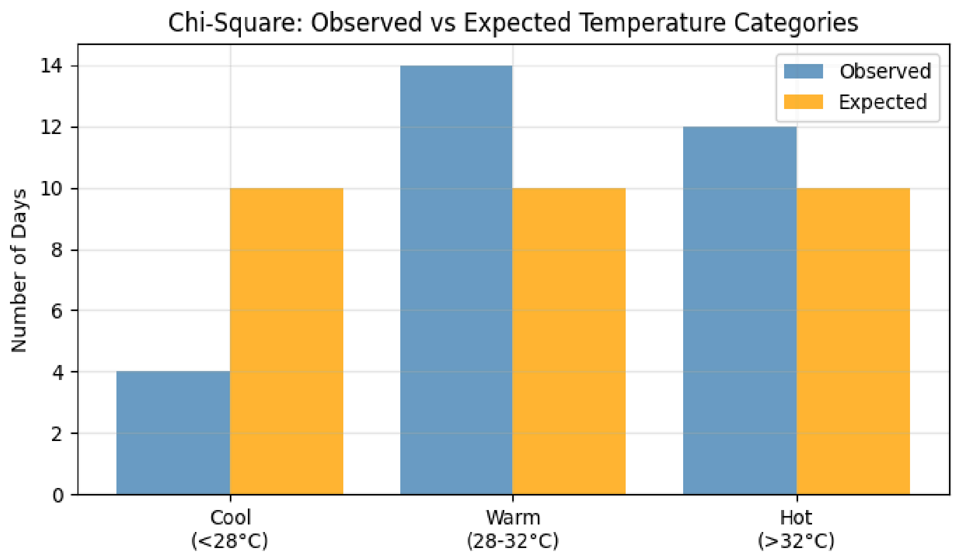

Figure 5: *Chi-square test — observed vs. expected day counts per temperature category.*

The computed chi-square statistic, 5.60, is slightly below the critical value of 5.9915 at df = 2 and $\alpha = 0.05$, and the corresponding p-value, 0.0608, is just above the 0.05 threshold. The null hypothesis of a uniform distribution across Cool, Warm, and Hot days is therefore not rejected at the conventional 5% significance level, although the result is close to the boundary. It is worth noting that the expected count of 10 per category is on the low side for the chi-square approximation to be fully reliable (a common rule of thumb recommends expected counts of at least 5, which is satisfied here, but only narrowly), so the test's statistical power with only n = 30 days is limited. Practically, the observed pattern — only 4 Cool days against 14 Warm and 12 Hot days — does suggest a real skew toward warmer categories; a larger sample would likely sharpen this borderline result in one direction or the other.

6.5 Animated Visualization

Figure 6 shows a single frame (Day 30) captured from the FuncAnimation-based animation, which reveals both temperature curves one day at a time with an on-screen readout of the current day's values. The full interactive animation is available by running the accompanying notebook; a static report can only show one frame at a time, but the snapshot below illustrates the final state of the animation after all 30 days have been plotted.

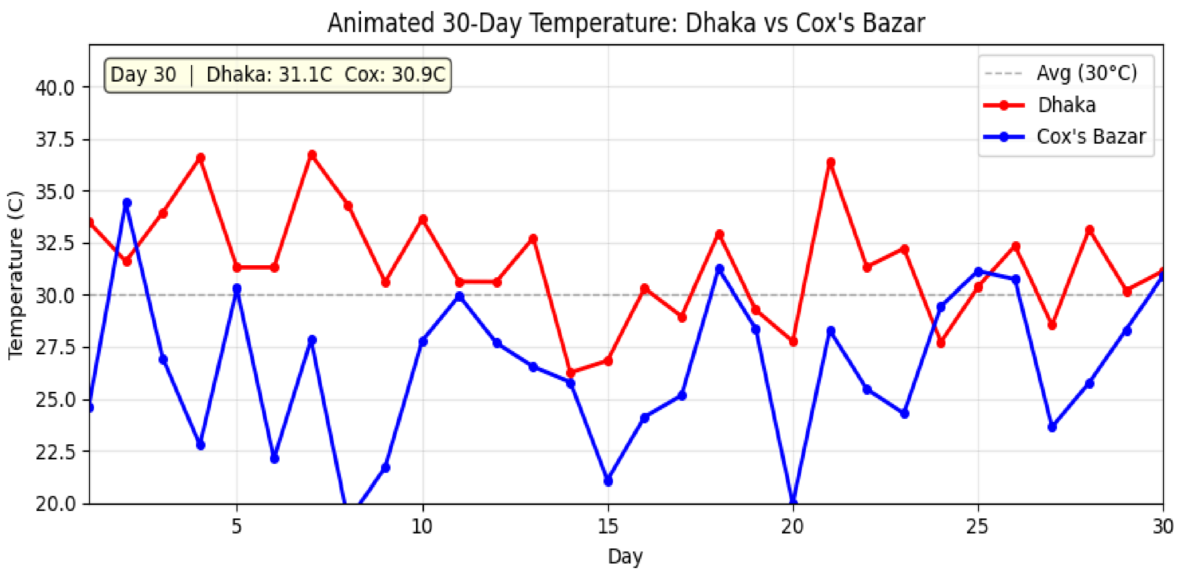

Figure 6: *The animated 30-day temperature plot.*

6.6 Discussion

Taken together, the three hypothesis tests paint a coherent picture. Both the one-sample and two-sample t-tests reject their respective null hypotheses with strong statistical significance, providing solid evidence that, in this simulation, Dhaka is hotter than both the 30°C national reference and Cox's Bazar. This is expected by construction, since Dhaka's generating mean (32°C) is set higher than Cox's Bazar's (27°C) and higher than the 30°C reference — but the value of the exercise is in confirming that a sample of only 30 days, despite its inherent randomness, is large enough for that designed difference to be detected reliably by a formal test.

The chi-square test result is more nuanced: it fails to reject uniformity at the 5% level, but the p-value of 0.0608 sits close enough to the threshold that the conclusion should be treated cautiously rather than as firm evidence that Dhaka's temperatures are evenly spread across categories. This is a useful reminder that "fail to reject H_0" is not the same as "proof that H_0 is true" — it may simply reflect limited statistical power at this small sample size and category resolution.

7. Conclusion

This project successfully demonstrated how simulation and statistical hypothesis testing can be combined in Python to analyse synthetic weather data. Thirty days of daily temperature data were generated for Dhaka and Cox's Bazar using independent Normal distributions, and three classical hypothesis tests were applied to the resulting series.

The one-sample t-test showed that Dhaka's simulated mean temperature (31.44°C) is significantly higher than the 30°C national reference ($t = 2.9122$, $df = 29$, $p = 0.0068$), and the two-sample t-test showed that Dhaka is significantly hotter than Cox's Bazar ($t = 5.8583$, $df = 58$, $p < 0.0001$, mean difference = 4.92°C). The chi-square goodness-of-fit test, by contrast, did not reject the assumption that Dhaka's Cool/Warm/Hot days are uniformly distributed ($\chi^2 = 5.60$, $df = 2$, $p = 0.0608$), though the result was close to the significance boundary and pointed toward a real skew in favour of warmer days that a larger sample could confirm.

Beyond the specific numerical results, the project illustrates the broader value of Python's scientific computing stack — NumPy, SciPy, and Matplotlib — for rapidly prototyping a full simulate-test-visualize pipeline, and it highlights an important statistical lesson: failing to reject a null hypothesis is not equivalent to proving it true, particularly with small samples and borderline p-values.

7.1 Limitations

- The dataset is entirely simulated and is not calibrated against real meteorological station records.
- The analysis is limited to a single variable (temperature) over a short, fixed 30-day window.
- The chi-square test operates with a relatively small expected cell count (10 per category), limiting its statistical power.

7.2 Future Improvements

- Calibrate the simulation against real historical weather-station data for Dhaka and Cox's Bazar.
- Extend the model toward a Richardson-style multivariate weather generator that also simulates rainfall and humidity (see Literature Review).
- Integrate a real-time weather API to compare simulated and observed conditions.
- Explore machine-learning-based forecasting models alongside the classical hypothesis-testing approach.
- Extend the comparison to additional cities for a multi-city analysis.

8. References

[1] C. W. Richardson, “Stochastic simulation of daily precipitation, temperature, and solar radiation,” Water Resources Research, vol. 17, no. 1, pp. 182–190, 1981.

[2] D. S. Wilks and R. L. Wilby, “The weather generation game: A review of stochastic weather models,” Progress in Physical Geography, vol. 23, no. 3, pp. 329–357, 1999.

[3] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[4] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.

[5] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[6] D. C. Montgomery and G. C. Runger, Applied Statistics and Probability for Engineers, 7th ed. Hoboken, NJ: Wiley, 2018.

[7] NumPy Developers, “numpy.random.Generator.normal — NumPy v2 Manual.” https://numpy.org/doc/stable/reference/random/generated/numpy.random.normal.html

[8] SciPy Developers, “scipy.stats — Statistical functions — SciPy Manual.” https://docs.scipy.org/doc/scipy/reference/stats.html

9. GitHub Repository for the Project

Link :

https://github.com/sohancse53/7th-semester/tree/main/Simulation%20%26%20Modeling%20Lab/project